Wirtschaftliche Konsequenzen eines effizienten SAT-Solvers

Entscheidungstheorie, Auktionen, Märkte, Produktion und Sicherheit

Stephan Epp
M.Sc.

12. Oktober 2026

Zusammenfassung

Die Grundarbeit zeigt, dass sich SAT über die Kette 3-SAT → Clique → Subgraph auf ein Subgraph-Orakel zurückführen lässt, und ordnet die Konsequenzen komplexitätstheoretisch ein. Die vorliegenden Ergänzungskapitel leiten daraus Konsequenzen für die Wirtschaft ab. Ausgangspunkt ist die Beobachtung, dass viele ökonomische Entscheidungsprobleme (Zuteilung, Planung, Konfiguration, Auktionsauswertung, Gleichgewichtsberechnung) in NP liegen und deshalb auf SAT reduzierbar sind. Alle Aussagen werden ausdrücklich *bedingt* formuliert: Sie gelten unter der Hypothese, dass ein exakter SAT-Entscheider mit Laufzeit $T(N)$ existiert (polynomiell $\Leftrightarrow$ P = NP). Die Arbeit liefert vollständige Beweise für die Reduktionen und Kodierungen, ordnet Aussagen nach ihrem Status (unbedingt, konditional, zitiert) und benennt die Grenzen: Zählprobleme, Wissens- und Kommunikationsengpässe, Anreizprobleme und die kryptographischen Nebenwirkungen bleiben bzw. entstehen.

Inhaltsverzeichnis

1 Einleitung, Hypothesen und Geltungsbereich **3**
- 1.1 Fragestellung . . . 3
- 1.2 Effizienzhypothesen . . . 3
- 1.3 Kennzeichnung der Aussagen . . . 4
- 1.4 Aufbau . . . 4

2 Formales Fundament **4**
- 2.1 Wirtschaftliche Optimierungsprobleme . . . 4
- 2.2 Tseitin-Kodierung, Vergleicher und Summierer . . . 5
- 2.3 Von der Entscheidung zur Lösung und Optimierung . . . 6
- 2.4 Äquivalenz der Hypothese mit P = NP . . . 7

3 Entscheidungstheorie **8**
- 3.1 Entscheidung unter Sicherheit . . . 8
- 3.2 Entscheidung unter Risiko und Unsicherheit . . . 9

4 Kombinatorische Auktionen und Mechanismen **10**
4.1 Vickrey-Clarke-Groves-Mechanismus 11

5 Frequenzzuteilung und Tauschmärkte **11**
5.1 Station Repacking (FCC-Anreizauktion) 11
5.2 Tauschmärkte (Nierenaustausch) 12

6 Produktion, Logistik und Konfiguration **12**
6.1 Terminplanung 12
6.2 Produktkonfiguration 12
6.3 Standortplanung 13

7 Spiele, Gleichgewichte und kollektive Entscheidungen **13**
7.1 Soziale Wahl 13

8 Kryptographie, Zahlungsverkehr und Sicherheit **14**

9 Wohlfahrtsmodell und Informationsgrenzen **14**

10 Experimente mit Subgraph-SAT-Solver **15**
10.1 Ziel und Versuchsaufbau 15
10.2 Instanzfamilien 16
10.3 Messwerte 16
10.4 Interpretation und Grenzen 20

11 Grenzen, Messplan und Fazit **20**
11.1 Messplan mit dem Repository 21
11.2 Fazit 21

1 Einleitung, Hypothesen und Geltungsbereich

1.1 Fragestellung

Die Grundarbeit behandelt die Frage, welche Entscheidungsprobleme sich über boolesche Formeln in die Subgraph-Frage $K_m \sqsubseteq G_\varphi$ übersetzen lassen, und beweist die Korrektheit der Kette 3-SAT $\to$ Clique $\to$ Subgraph (Abschnitt „Formale Reduktionskette und Beweise“). Im Abschnitt „Wirtschaftliche Bedeutung“ der Grundarbeit wird die ökonomische Relevanz nur qualitativ erwähnt. Hier wird sie formalisiert. Die leitende Beobachtung ist elementar: Ein Entscheidungsträger (Unternehmen, Behörde, Marktplatz) wählt aus einer endlichen, aber kombinatorisch großen Menge zulässiger Alternativen. Ist die Zulässigkeit und die Bewertung einer Alternative effizient prüfbar, so liegt die zugehörige Entscheidungsfrage in NP und ist damit auf SAT reduzierbar. Ein effizienter SAT-Entscheider löst dann *jede* derartige Frage mit polynomiellem Zusatzaufwand.

1.2 Effizienzhypothesen

Die Grundarbeit unterscheidet zwischen einem heuristischen Signaturorakel und dem exakten Entscheider (`ExactSubgraph`) und hält fest, dass nur letzterer die Korrektheit der Reduktion garantiert. Ob ein *exakter* Entscheider für beliebige Eingaben in polynomieller Zeit arbeitet, wird in der Grundarbeit nicht bewiesen. Aus diesem Grund werden hier Hypothesen explizit benannt und alle wirtschaftlichen Folgerungen als Implikationen formuliert.

Die *Größe* $|\varphi|$ einer CNF-Formel sei die Gesamtzahl der Literalvorkommen (die Summe der Klauselängen); es gilt $|\varphi| \geq$ Klauselzahl.

Hypothese 1.1 (Effizienzhypothesen)**.** Sei $T : \mathbb{N} \to \mathbb{N}$ monoton wachsend.

(a) (E_T): Es gibt einen deterministischen Algorithmus, der für jede CNF-Formel φ mit $|\varphi| = N$ in höchstens $T(N)$ Schritten korrekt entscheidet, ob φ erfüllbar ist.

(b) ($\mathrm{E}_{\mathrm{poly}}$): ($\mathrm{E}_T$) gilt für ein Polynom T.

(c) (S_3): Es gibt einen deterministischen Algorithmus, der für jeden ungerichteten Graphen G und jedes k in höchstens $c \cdot |V(G)|^3$ Schritten (c konstant) korrekt entscheidet, ob $K_k \sqsubseteq G$ (also ob G eine k-Clique enthält).

Bemerkung 1.2 (Status der Hypothesen)**.** (S_3) ist die Hypothese, dass ein exaktes Subgraph-Orakel mit der in der README des Repositories genannten kubischen Laufzeit auf allen Clique-Instanzen korrekt arbeitet. Wegen Satz 2.14 und Lemma 2.8 gilt (S_3) $\Rightarrow$ ($\mathrm{E}_{\mathrm{poly}}$) $\Leftrightarrow$ P = NP. (S_3) ist daher eine Hypothese von der Stärke eines Beweises von P = NP mit expliziten Exponenten. Das Problem ist seit 1971 offen [1, 5, 6]. Weder die Grundarbeit noch dieses Dokument beweisen eine der Hypothesen; sie werden als Annahmen verwendet. Die Rolle der Hypothesen ist doppelt: Als *Szenario* erlauben sie, den ökonomischen Wert eines effizienten Solvers präzise zu beziffern. Als *Prüfstein* liefern sie Kontraposition: Ist eine Folgerung ($\mathrm{E}_{\mathrm{poly}}$) $\Rightarrow X$ mit einer als falsch geltenden Aussage X bewiesen, so spricht das gegen die Hypothese.

1.3 Kennzeichnung der Aussagen

Jeder Satz, jedes Lemma und jedes Korollar trägt eine der folgenden Marken:

- [U] *unbedingt*: rein mathematisch bewiesen, ohne Hypothese (Modell- und Kodierungsaussagen, Reduktionen, Korrektheitsbeweise);
- [K] *konditional*: gilt unter der angegebenen Effizienzhypothese;
- [Z] *zitiert*: Ergebnis der Literatur, ohne Beweis übernommen und mit Quelle versehen.

Beweise werden für alle Aussagen der Typen [U] und [K] vollständig geführt.

1.4 Aufbau

Kapitel 2 legt das formale Fundament (Tseitin-Kodierung, Selbstreduzierbarkeit, Optimierung aus Entscheidung). Kapitel 3 behandelt die Entscheidungstheorie unter Sicherheit, Risiko und Unsicherheit. Kapitel 4 behandelt kombinatorische Auktionen und Mechanismen, Kapitel 5 Märkte mit Nebenbedingungen (Frequenzzuteilung, Tauschmärkte). Kapitel 6 betrifft Produktion, Logistik und Konfiguration. Kapitel 7 behandelt Spiele, Gleichgewichte und kollektive Entscheidungen, Kapitel 8 die Folgen für Kryptographie und Zahlungsverkehr. Kapitel 9 bildet ein gesamtwirtschaftliches Wohlfahrtsmodell, Kapitel 11 grenzt ab und gibt einen Messplan. Kapitel 10 enthält jeweils eine kurze Einführung in die ökonomische Problemklasse, die Reduktion auf SAT, die Kodierung der Zielfunktion und die experimentelle Auswertung.

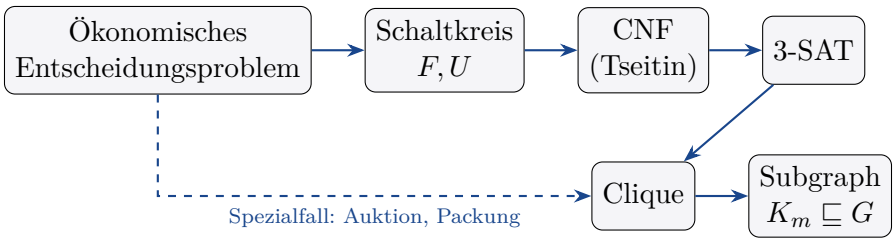


Abbildung 1: Reduktionspfad: Wirtschaftliche Entscheidungsprobleme münden über Tseitin-Kodierung in die Kette der Grundarbeit. Auktionsprobleme besitzen zusätzlich eine direkte Reduktion auf Clique (Satz 4.3).

2 Formales Fundament

2.1 Wirtschaftliche Optimierungsprobleme

Definition 2.1 (Wirtschaftliches Optimierungsproblem)**.** Ein *wirtschaftliches Optimierungsproblem* (WOP) Π ordnet jeder Instanz I (Binärstring) zu: eine Anzahl $n = n(I) \le \mathrm{poly}(|I|)$ von Entscheidungsbits, eine Breite $L = L(I) \le \mathrm{poly}(|I|)$, eine Zulässigkeitsfunktion $F_I : \{0,1\}^n \to \{0,1\}$ und eine Zielfunktion $U_I : \{0,1\}^n \to \{0, \dots, 2^L - 1\}$, beide in Zeit $\mathrm{poly}(|I|)$ auswertbar. Gesucht ist

$$\mathrm{opt}(I) = \max\{U_I(x) \mid x \in \{0,1\}^n,\ F_I(x) = 1\}$$

(oder $-\infty$, falls keine zulässige Lösung existiert). Die *Entscheidungsvariante* Π_{dec} fragt für ein gegebenes $B \in \{0, \dots, 2^L - 1\}$, ob ein x mit $F_I(x) = 1$ und $U_I(x) \geq B$ existiert. Minimierungsprobleme werden durch $U_I' = (2^L - 1) - U_I$ erfasst.

Negative oder rationale Zielwerte werden durch Verschiebung und Skalierung auf ganze Zahlen in $\{0, \dots, 2^L - 1\}$ zurückgeführt; die Bitbreite L wächst dabei höchstens polynomiell.

Lemma 2.2 (Mitgliedschaft in NP)**.** [U] Für jedes WOP gilt $\Pi_{\text{dec}} \in \text{NP}$.

Beweis. Zeuge ist $x \in \{0,1\}^n$ mit $n \leq \text{poly}(|I|)$. Die Prüfung $F_I(x) = 1 \wedge U_I(x) \geq B$ erfolgt nach Definition in Polynomialzeit. □

Satz 2.3 (Schaltkreisdarstellung)**.** [Z] Ist f in Polynomialzeit berechenbar, so existiert eine in Polynomialzeit konstruierbare Familie boolescher Schaltkreise (C_m) polynomieller Größe mit $C_{|x|}(x) = f(x)$ [3, 4].

Für jedes WOP gibt es somit Schaltkreise F_I und U_I der Größe $s \leq \text{poly}(|I|)$, die aus I in Polynomialzeit konstruierbar sind. In den Kapiteln 4–7 werden für die wichtigsten Probleme die Schaltkreise und Formeln jedoch *explizit* angegeben; das liefert konkrete Größen und erlaubt, die Kodierungen mit dem Solver des Repositories zu testen.

2.2 Tseitin-Kodierung, Vergleicher und Summierer

Ein Schaltkreis besteht aus Eingängen und Gattern AND, OR, XOR, NOT mit höchstens zwei Eingängen.

Lemma 2.4 (Tseitin)**.** [U] Zu jedem Schaltkreis C mit n Eingängen und s Gattern gibt es eine in Linearzeit konstruierbare CNF-Formel $\tau(C)$ über den Eingangsvariablen $x_1, \dots, x_n$ und je einer Variablen g_v pro Gatter v mit $|\tau(C)| \leq 12\,s$, sodass für jede Belegung $x \in \{0,1\}^n$ genau eine Fortsetzung auf die Gattervariablen existiert, die $\tau(C)$ erfüllt, und in dieser gilt $g_v =$ Wert von Gatter v bei Eingabe x.

Beweis. Pro Gatter v mit Eingängen a, b (Variablen oder Gattervariablen) werden die Klauseln der Äquivalenz $g_v \leftrightarrow f(a,b)$ hinzugefügt:

$$\begin{aligned}
\text{AND}: &\ (\neg g \vee a) \wedge (\neg g \vee b) \wedge (g \vee \neg a \vee \neg b),\\
\text{OR}: &\ (g \vee \neg a) \wedge (g \vee \neg b) \wedge (\neg g \vee a \vee b),\\
\text{XOR}: &\ (\neg g \vee a \vee b) \wedge (\neg g \vee \neg a \vee \neg b) \wedge (g \vee \neg a \vee b) \wedge (g \vee a \vee \neg b),\\
\text{NOT}: &\ (g \vee a) \wedge (\neg g \vee \neg a).
\end{aligned}$$

Jede dieser Klauselmengen ist genau dann erfüllt, wenn $g = f(a,b)$ gilt (Überprüfung der Wahrheitstafel: die Klauseln verbieten genau die Zeilen mit $g \neq f(a,b)$). Pro Gatter entstehen höchstens $4 \cdot 3 = 12$ Literalvorkommen. Sei x fest. Nach Induktion über eine topologische Ordnung der Gatter ist der Wert von g_v durch die Äquivalenzklauseln eindeutig festgelegt und gleich dem Gatterwert. Damit existiert genau eine erfüllende Fortsetzung. □

Lemma 2.5 (Vergleich mit einer Konstanten)**.** [U] Sei $U = (u_{L-1}, \dots, u_0)$ ein L-Bit-Wort (little-endian indiziert, u_i Gatterausgänge) und $B \in \{0, \dots, 2^L - 1\}$ eine Konstante mit Binärziffern b_i. Definiere $\text{ge}_{-1} = 1$ und

$$\text{ge}_i = \begin{cases} u_i \wedge \text{ge}_{i-1}, & b_i = 1,\\ u_i \vee \text{ge}_{i-1}, & b_i = 0. \end{cases}$$

Dann gilt $\mathrm{ge}_{L-1} = 1 \iff \mathrm{val}(U) \geq B$. Die Schaltung hat L Gatter.

Beweis. Behauptung $P(i)$: $\mathrm{ge}_i = 1$ genau dann, wenn die unteren $i+1$ Bits von U (als Zahl) mindestens so groß sind wie die unteren $i+1$ Bits von B. Für $i = -1$ (leere Bitmenge, beide Zahlen 0) ist dies wahr. Schritt: $\mathrm{val}_{\leq i}(U) \geq \mathrm{val}_{\leq i}(B)$ gilt genau dann, wenn $u_i > b_i$ oder ($u_i = b_i$ und $\mathrm{val}_{\leq i-1}(U) \geq \mathrm{val}_{\leq i-1}(B)$). Für $b_i = 1$ ist $u_i > b_i$ unmöglich und $u_i = b_i$ bedeutet $u_i = 1$; das ergibt $u_i \wedge \mathrm{ge}_{i-1}$. Für $b_i = 0$ bedeutet $u_i > b_i$ gerade $u_i = 1$, und $u_i = b_i$ bedeutet $u_i = 0$; das ergibt $u_i \vee \mathrm{ge}_{i-1}$. Mit $i = L-1$ folgt die Aussage. □

Lemma 2.6 (Summierer)**.** [U] Seien $y_1, \ldots, y_n$ Bitvariablen und $c_1, \ldots, c_n \in \{0, \ldots, 2^L - 1\}$ Konstanten. Mit $L' = L + \lceil \log_2(n+1) \rceil$ existiert ein Schaltkreis mit $O(nL')$ Gattern, dessen L'-Bit-Ausgang $\sum_j c_j y_j$ darstellt.

Beweis. Es gilt $0 \leq \sum_j c_j y_j \leq n(2^L - 1) < 2^{L'}$, die Summe passt also in L' Bit. Der Schaltkreis akkumuliert $A_0 = 0$, $A_j = A_{j-1} + c_j y_j$. Das Wort $c_j y_j$ hat in Bitposition i den Wert y_j falls c_j dort eine 1 trägt, sonst die Konstante 0; es kostet kein Gatter. Die Addition zweier L'-Bit-Worte erfolgt durch einen Ripple-Carry-Addierer aus L' Volladdierern (Summenbit $a \oplus b \oplus c$, Übertrag $(a \wedge b) \vee (c \wedge (a \oplus b))$; jeder Volladdierer benötigt 5 Gatter). Insgesamt entstehen $\leq 5nL'$ Gatter. Die Korrektheit folgt aus der Korrektheit des Volladdierers per Induktion über die Bitpositionen. □

Lemma 2.7 (Konversion nach 3-CNF)**.** [U] Zu jeder CNF φ mit $|\varphi| = N$ gibt es in Linearzeit eine erfüllbarkeitsäquivalente 3-CNF φ' mit höchstens N Klauseln und $|\varphi'| \leq 3N$; jede erfüllende Belegung von φ' liefert durch Einschränkung eine erfüllende Belegung von φ.

Beweis. Eine Klausel $(\ell_1 \vee \cdots \vee \ell_k)$ mit $k \geq 4$ wird durch $(\ell_1 \vee \ell_2 \vee z_1) \wedge (\neg z_1 \vee \ell_3 \vee z_2) \wedge \cdots \wedge (\neg z_{k-3} \vee \ell_{k-1} \vee \ell_k)$ mit frischen Variablen z_i ersetzt, das sind $k-2$ Klauseln der Länge ≤ 3. Erfüllt eine Belegung die ursprüngliche Klausel, etwa durch ℓ_j, so setze $z_i = 1$ für $i < j-1$ und $z_i = 0$ für $i \geq j-1$; alle neuen Klauseln sind erfüllt. Erfüllt umgekehrt eine Belegung alle neuen Klauseln und wäre jedes ℓ_i falsch, so erzwingt die erste Klausel $z_1 = 1$, die zweite dann $z_2 = 1$ usw. bis zur letzten Klausel $(\neg z_{k-3} \vee \ell_{k-1} \vee \ell_k)$, die dann verletzt wäre; Widerspruch. Die Klauselzahl ist $\sum_c \max(1, k_c - 2) \leq N$, die Literalzahl $\leq 3 \sum_c \max(1, k_c - 2) \leq 3N$. □

Lemma 2.8 (Laufzeit der Grundarbeit-Kette unter (S_3))**.** [K] Unter (S_3) kann die Erfüllbarkeit jeder CNF φ mit $|\varphi| = N$ in Zeit $O(N^3)$ entschieden werden. Insbesondere gilt (S_3) $\Rightarrow$ ($\mathrm{E}_{\mathrm{poly}}$) mit $T(N) = O(N^3)$.

Beweis. Nach Lemma 2.7 entsteht eine 3-CNF φ' mit $m \leq N$ Klauseln und $\leq 3N$ Literalvorkommen. Der Clique-Graph $G_{\varphi'}$ der Grundarbeit (Satz „3-SAT auf Clique“) hat einen Knoten pro Literalvorkommen, also $\leq 3N$ Knoten, und lässt sich in Zeit $O(N^2)$ konstruieren (paarweise Prüfung von Klauselindex und Komplementarität). Nach der Grundarbeit gilt φ' erfüllbar $\iff K_m \sqsubseteq G_{\varphi'}$. Nach ($\mathrm{S}_3$) wird dies in Zeit $c(3N)^3 = 27cN^3$ entschieden. Die Gesamtzeit ist $O(N^3)$. □

2.3 Von der Entscheidung zur Lösung und Optimierung

Satz 2.9 (Reduktionslemma)**.** [K] Gelte (E_T). Sei Π ein Entscheidungsproblem und $I \mapsto \varphi_I$ eine in Zeit $r(|I|)$ berechenbare Abbildung mit $I \in \Pi \iff \varphi_I \in \mathrm{SAT}$ und $|\varphi_I| \leq q(|I|)$, q monoton. Dann ist Π in Zeit $r(|I|) + T(q(|I|))$ entscheidbar. Ist T polynomiell, so ist $\Pi \in \mathrm{P}$.

Beweis. Berechne φ_I und wende den Entscheider an; die Größenschranke und die Monotonie von T liefern die Laufzeit. □

Satz 2.10 (Selbstreduzierbarkeit: Entscheiden ⇒ Finden)**.** [K] Gelte (E_T). Für jede CNF φ mit n Variablen und $|\varphi| = N$ lässt sich in höchstens $n+1$ Aufrufen des Entscheiders auf Formeln der Größe $\leq N$ und Zusatzzeit $O(nN)$ entweder Unerfüllbarkeit feststellen oder eine erfüllende Belegung berechnen.

Beweis. Ein Aufruf auf φ klärt Erfüllbarkeit. Ist φ erfüllbar, sei $\varphi_0 = \varphi$. Für $i = 1, \ldots, n$: Bilde $\varphi_{i-1}[x_i{:=}0]$ (Klauseln mit wahrem Literal entfernen, falsche Literale streichen; eine leere Klausel bedeutet unerfüllbar). Ist diese Formel erfüllbar, setze $\varphi_i = \varphi_{i-1}[x_i{:=}0]$, sonst $\varphi_i = \varphi_{i-1}[x_i{:=}1]$. Invariante: φ_i ist erfüllbar. Für $i = 0$ nach Voraussetzung. Ist $\varphi_{i-1}[x_i{:=}0]$ unerfüllbar, so muss die erfüllbare Formel φ_{i-1} eine erfüllende Belegung mit $x_i = 1$ besitzen, also ist $\varphi_{i-1}[x_i{:=}1]$ erfüllbar. Nach n Schritten ist alles belegt, und die gesammelte Belegung erfüllt φ. Die Substitutionen vergrößern die Formel nie. □

Satz 2.11 (Optimierung aus Entscheidung)**.** [K] Gelte (E_T). Sei ein WOP-Instanz mit Schaltkreisen F, U der Gesamtgröße s, n Entscheidungsbits und Zielbreite L gegeben. Dann lassen sich mit höchstens $L + n + 2$ Aufrufen des Entscheiders auf Formeln der Größe $\leq 12(s+L)+1$ (a) die Zulässigkeit feststellen, (b) der Optimalwert opt bestimmen und (c) ein optimaler Entscheidungsvektor $x^\star$ konstruieren. Die Gesamtzeit beträgt $(L + n + 2)\,T(12(s+L)+1) + O(n\,(s+L))$.

Beweis. Für $B \in \{0, \ldots, 2^L - 1\}$ sei $\varphi_B = \tau(F) \wedge \tau(\mathrm{GE}_B(U)) \wedge (g_F) \wedge (g_{\mathrm{GE}})$ mit dem Vergleicher aus Lemma 2.5. Nach Lemma 2.4 ist φ_B genau dann erfüllbar, wenn $\exists x : F(x) = 1 \wedge U(x) \geq B$. Die Größe ist $\leq 12(s+L)+2$ (die beiden Einheitsklauseln inbegriffen; die Schranke im Satz ist grob angegeben). (a) φ_0 ist genau dann erfüllbar, wenn eine zulässige Lösung existiert, da $U \geq 0$. (b) Ist φ_0 erfüllbar, so gilt $\varphi_B \in \mathrm{SAT} \iff B \leq \mathrm{opt}$ (Monotonie in B). Binäre Suche über $\{0, \ldots, 2^L - 1\}$ mit den Invarianten „$\varphi_{lo} \in \mathrm{SAT}$“ und „$hi = 2^L$ oder $\varphi_{hi} \notin \mathrm{SAT}$“, Start $lo = 0$, $hi = 2^L$, halbiert die Intervalllänge $hi - lo$ bei jedem Schritt exakt und benötigt daher $\leq L$ weitere Aufrufe; am Ende ist $lo = \mathrm{opt}$. (c) Wende Satz 2.10 auf φ_{opt} an; die Eingangsvariablen der Belegung bilden ein optimales $x^\star$. Zusammen: $1 + L + (n+1)$ Aufrufe, wobei der erste Aufruf von (c) mit einem Aufruf aus (b) zusammenfällt; $L + n + 2$ ist eine sichere obere Schranke. □

Korollar 2.12 (Polynomielle Lösbarkeit aller WOP)**.** [K] Gilt ($\mathrm{E}_{\mathrm{poly}}$), so ist jedes WOP (Optimalwert und Optimallösung) in Polynomialzeit lösbar. Insbesondere hat die Zahl der Entscheideraufrufe die Form $\mathrm{poly}(|I|)$.

Beweis. Nach Satz 2.3 sind F, U mit Größe $s \leq \mathrm{poly}(|I|)$ konstruierbar, $L, n \leq \mathrm{poly}(|I|)$. Satz 2.11 liefert die Behauptung. □

2.4 Äquivalenz der Hypothese mit P = NP

Satz 2.13. [Z] SAT ist NP-vollständig (Cook 1971, Levin 1973; [1, 2]).

Satz 2.14 ($(\mathrm{E}_{\mathrm{poly}}) \Leftrightarrow \mathrm{P} = \mathrm{NP}$)**.** [U] ($\mathrm{E}_{\mathrm{poly}}$) gilt genau dann, wenn P = NP.

Beweis. ($\Rightarrow$) Sei $L \in \mathrm{NP}$. Nach Satz 2.13 gibt es eine Polynomialzeitreduktion $L \leq_p \mathrm{SAT}$; mit ($\mathrm{E}_{\mathrm{poly}}$) ist $\mathrm{SAT} \in \mathrm{P}$ (die Entscheidung einer CNF der Größe N in Zeit $\mathrm{poly}(N)$ ist Polynomialzeit), also $L \in \mathrm{P}$. Wegen $\mathrm{P} \subseteq \mathrm{NP}$ folgt P = NP. ($\Leftarrow$) $\mathrm{SAT} \in \mathrm{NP} = \mathrm{P}$; die Größe im Sinne von $|\varphi|$ ist polynomiell äquivalent zur Eingabelänge. □

Satz 2.15 (P = NP ⇒ FP = FNP)**.** [K] Gelte (E_{poly}). Sei $R \subseteq \{0,1\}^* \times \{0,1\}^*$ in Polynomialzeit entscheidbar und polynomiell beschränkt, d. h. $R(x,w) \Rightarrow |w| \leq p(|x|)$. Dann gibt es einen Polynomialzeitalgorithmus, der zu jedem x ein w mit $R(x,w)$ ausgibt, falls ein solches existiert, und sonst „keines“ meldet.

Beweis. Die Sprache $L_R = \{(x,u) \mid \exists v : R(x,uv)\}$ liegt in NP (Zeuge v mit $|uv| \leq p(|x|)$), also nach Satz 2.14 in P. Teste $(x,\varepsilon) \in L_R$; falls nicht, melde „keines“. Sonst setze $u = \varepsilon$ und wiederhole höchstens $p(|x|)$ mal: Gilt $R(x,u)$, gib u aus. Andernfalls ist $(x,u) \in L_R$ und $R(x,u)$ falsch, d. h. es gibt eine echte Verlängerung uv mit $R(x,uv)$, $v \neq \varepsilon$; setze $u := u0$, falls $(x,u0) \in L_R$, sonst $u := u1$. Die Invariante $(x,u) \in L_R$ bleibt erhalten, und $|u|$ wächst um 1; wegen $|w| \leq p(|x|)$ endet das Verfahren spätestens nach $p(|x|)$ Schritten mit einem Zeugen. □

3 Entscheidungstheorie

3.1 Entscheidung unter Sicherheit

Satz 3.1 (Nutzendarstellung auf endlichen Mengen)**.** [U] Sei $\succeq$ eine vollständige, transitive Präferenzrelation auf einer endlichen Menge A. Dann ist $u(a) = |\{b \in A \mid a \succeq b\}|$ eine Nutzenfunktion: $a \succeq c \iff u(a) \geq u(c)$.

Beweis. Sei $D(a) = \{b : a \succeq b\}$. Gilt $a \succeq c$, so folgt aus $c \succeq b$ stets $a \succeq b$ (Transitivität), also $D(c) \subseteq D(a)$ und $u(a) \geq u(c)$. Gilt $a \not\succeq c$, so ist wegen Vollständigkeit $c \succeq a$, nach dem eben Gezeigten (mit vertauschten Rollen) $D(a) \subseteq D(c)$, und $c \in D(c) \setminus D(a)$ (Reflexivität folgt aus Vollständigkeit). Also $u(c) > u(a)$. Damit gilt $a \succeq c \iff u(a) \geq u(c)$. □

Satz 3.2 (Rationale Wahl mit implizit gegebener Alternativenmenge)**.** [K] Sei $A = \{x \in \{0,1\}^n \mid F(x) = 1\}$ und der Nutzen $U : \{0,1\}^n \to \{0,\ldots,2^L - 1\}$ durch Schaltkreise der Größe s gegeben. Gilt (E_T), so wird ein nutzenmaximales Element in $(L + n + 2)$ Entscheideraufrufen gefunden (Satz 2.11); unter (E_{poly}) in Polynomialzeit.

Beweis. Satz 2.11. □

Bemerkung 3.3 (Präferenz nur als Vergleich gegeben)**.** Ist die Präferenz nur als Schaltkreis $P(a,b)$ („$a \succeq b$“) gegeben, lautet die Frage nach einem Maximum $\exists a \forall b : P(a,b)$ und liegt in Σ_2^p. Mit nur einem SAT-Orakel ist sie nicht direkt kodierbar; erst unter (E_{poly}) wird sie polynomiell (Satz 3.10). Gibt es eine Nutzenfunktion als Schaltkreis, entfällt das Problem.

Satz 3.4 (Lexikographische Optimierung)**.** [K] Seien $U_1,\ldots,U_k$ Zielfunktionen (Priorität $U_1 \succ \cdots \succ U_k$). Setze $F_0 = F$ und $v_i = \max\{U_i(x) \mid F_{i-1}(x) = 1\}$, $F_i = F_{i-1} \wedge [U_i \geq v_i]$. Dann ist x genau dann lexikographisch optimal, wenn $F_k(x) = 1$. Dies kostet k Optimierungen im Sinne von Satz 2.11.

Beweis. Induktion: $F_i(x) = 1$ gilt genau dann, wenn x zulässig ist und $(U_1,\ldots,U_i)(x)$ lexikographisch maximal unter den zulässigen Punkten ist. Schritt: Unter den Punkten mit $F_{i-1} = 1$ (alle haben dieselben Werte $v_1,\ldots,v_{i-1}$) sind genau diejenigen mit maximalem $U_i = v_i$ lexikographisch optimal bis Stufe i. □

Satz 3.5 (Aufzählung der Pareto-Front)**.** [K] Seien $U_1, \ldots, U_k$ gegeben, $U = (U_1, \ldots, U_k)$. Starte mit $\mathcal{Y} = \emptyset$. Wiederhole: Maximiere $\sum_i U_i(x)$ über $\{x : F(x) = 1,\ \forall y \in \mathcal{Y}\ \exists i : U_i(x) > y_i\}$; ist diese Menge leer, stoppe; sonst füge $U(x^\star)$ zu $\mathcal{Y}$ hinzu. Dann ist $\mathcal{Y}$ am Ende genau die Menge der Pareto-optimalen Zielvektoren P, und es werden $|P| + 1$ Optimierungen (Satz 2.11) benötigt.

Beweis. Sei R die Restmenge (nicht schwach von einem $y \in \mathcal{Y}$ dominierte Punkte). *Jeder gefundene Vektor ist Pareto-optimal:* Dominiert z den Maximierer $x^\star$ ($U(z) \geq U(x^\star), \neq$), so ist die Summe größer. Wäre $U(z) \leq y$ für ein $y \in \mathcal{Y}$, so $U(x^\star) \leq y$, im Widerspruch zu $x^\star \in R$. Also $z \in R$ mit größerer Summe, Widerspruch. *Jedes $p \in P$ wird gefunden:* Ist p nicht gefunden und $p \leq y$ für ein gefundenes y, so folgt $p = y$ (sonst dominierte y das Pareto-optimale p); also $p \in R$, und $R \neq \emptyset$. Jede Runde fügt einen neuen Vektor hinzu, $|P|$ Runden plus eine leere Schlussrunde. □

Satz 3.6 (Soziale Wohlfahrt)**.** [K] Sind $U_1, \ldots, U_k$ individuelle Nutzen (Schaltkreise) und $W : \mathbb{Z}^k \to \mathbb{Z}$ in Polynomialzeit berechenbar (z. B. utilitaristisch $\sum U_i$, egalitär $\min_i U_i$, Nash-Produkt $\prod U_i$), so ist $\max_x W(U_1(x), \ldots, U_k(x))$ ein WOP; unter ($\mathrm{E_{poly}}$) ist es in Polynomialzeit lösbar.

Beweis. $W \circ U$ ist polynomiell berechenbar mit Bitbreite $\leq kL$ (Produkt) bzw. $L + \log k$ (Summe); Kor. 2.12. □

3.2 Entscheidung unter Risiko und Unsicherheit

Satz 3.7 (Erwartungsnutzen bei explizitem Zustandsraum)**.** [K]Sind Zustände $s = 1..m$ mit rationalen Wahrscheinlichkeiten π_s explizit aufgezählt, so ist $x \mapsto \sum_s \pi_s U(x, s)$ polynomiell berechenbar (nach Skalierung ganzzahlig); die Maximierung ist ein WOP. Dasselbe gilt für Maximin $\min_s U(x, s)$ (Wald) und Minimax-Regret.

Satz 3.8 (Grenze: implizite Zustandsräume)**.** [Z] Die Berechnung von $\Pr_{s \in \{0,1\}^m}[\varphi(s)] = \#\varphi / 2^m$ für CNF φ ist #P-vollständig [7]. Es ist nicht bekannt, dass ($\mathrm{E_{poly}}$) daraus polynomielle Lösbarkeit folgert. Näherungszählung liegt in $\mathrm{BPP^{NP}}$ [8].

Ein SAT-Entscheider beantwortet Existenzfragen, *nicht* Erwartungswerte über exponentiell vielen Zuständen. Erwartungsnutzenmaximierung mit implizitem Zustandsraum bleibt daher außerhalb der Reichweite der Ergebnisse dieser Arbeit.

Satz 3.9 (Robuste Zweistufenentscheidung ist in Σ_2^p)**.** [U] Gegeben Schaltkreise $U(a, s)$ und B; die Frage $\exists a\, \forall s \in \{0,1\}^m :\ U(a, s) \geq B$ liegt in Σ_2^p.

Beweis. Zeuge a polynomieller Länge; das Prädikat $\forall s : U(a, s) \geq B$ ist in coNP. □

Satz 3.10 (Kollaps der Polynomialzeithierarchie)**.** [K] Gilt ($\mathrm{E_{poly}}$), so ist $\Sigma_k^p = \Pi_k^p = \mathrm{P}$ für alle k, d. h. PH = P. Insbesondere sind robuste Mehrstufenprobleme mit polynomiell codierter Unsicherheitsmenge in Polynomialzeit lösbar.

Beweis. Induktion über k. $k = 1$: $\Sigma_1^p = \mathrm{NP} = \mathrm{P}$ (Satz 2.14), $\Pi_1^p = \mathrm{coNP} = \mathrm{coP} = \mathrm{P}$. Schritt: $L \in \Sigma_{k+1}^p$ heißt $x \in L \iff \exists y\ R(x, y)$ mit $R \in \Pi_k^p = \mathrm{P}$ (Induktion); also $L \in \mathrm{NP} = \mathrm{P}$. Dann $\Pi_{k+1}^p = \mathrm{co}\Sigma_{k+1}^p = \mathrm{coP} = \mathrm{P}$. □

Satz 3.11 (Gegenbeispielgesteuerte Lösung mit SAT)**.** [U] Algorithmus: $S := \emptyset$ (Szenarien). (1) Finde a mit $U(a, s) \geq B\ \forall s \in S$ (SAT; sonst: kein robustes a). (2) Suche s mit $U(a, s) < B$ (SAT; sonst: a robust, Ende). (3) $S := S \cup \{s\}$, weiter bei (1). Der Algorithmus ist korrekt und terminiert nach höchstens 2^m Runden.

Beweis. Ende in (2) bedeutet $\forall s : U(a,s) \geq B$. Unerfüllbarkeit in (1) bedeutet: Jedes a verletzt ein Szenario aus $S \subseteq \{0,1\}^m$, also kein robustes a. Das gefundene s liegt nicht in S, da a alle $s \in S$ erfüllt; $|S|$ wächst je Runde um 1 und ist $\leq 2^m$. □

Bemerkung 3.12 (Sequentielle Entscheidungen)**.** Vollständig beobachtbare MDP mit expliziten Zuständen sind in P lösbar (Rückwärtsinduktion) und profitieren nicht von SAT; endliche partiell beobachtbare MDP sind PSPACE-vollständig [9]. P = NP ändert nichts an PSPACE (Tabelle 2).

Bemerkung 3.13 (Begrenzte Rationalität)**.** Simons Kritik [10] beruht auf begrenzten Rechenressourcen. Unter ($\mathrm{E}_{\mathrm{poly}}$) verschwindet für alle NP-Entscheidungsprobleme die Lücke zwischen „rationaler“ und „berechenbarer“ Wahl; sie bleibt für Erwartungswerte, PSPACE-Probleme und Informationsmängel bestehen. Dies ist eine Deutung, kein Satz.

4 Kombinatorische Auktionen und Mechanismen

Definition 4.1 (Winner Determination Problem, WDP)**.** Gegeben Güter $M = \{1..m\}$, Gebote (S_j, p_j), $S_j \subseteq M$, $p_j \in \mathbb{N}$, $j = 1..n$. Zulässig ist $J \subseteq [n]$ mit $S_j \cap S_k = \emptyset$ für $j \neq k \in J$. Erlös $\sum_{j \in J} p_j$. WDP-Dec: Existiert zulässiges J mit Erlös $\geq B$?

Satz 4.2 (NP-Vollständigkeit)**.** [U] WDP-Dec ist NP-vollständig (vgl. [11, 12]).

Beweis. In NP (Zeuge J). Reduktion von Independent Set $(G = (V,E), k$; NP-vollständig [2]): Güter $E \cup \{v^0 : v \in V\}$, je Knoten v ein Gebot $S_v = \{e \in E : v \in e\} \cup \{v^0\}$, $p_v = 1$. Zwei Gebote u, v sind genau dann konfliktfrei, wenn $\{u,v\} \notin E$ (die privaten Güter stören nicht). Also ist J genau dann zulässig, wenn J unabhängig ist; Erlös $= |J|$. □

Satz 4.3 (WDP → Clique → Subgraph)**.** [U] Der *Kompatibilitätsgraph* C hat die Knotenmenge $[n]$ und die Kante $\{j,k\}$ genau dann, wenn $S_j \cap S_k = \emptyset$. (a) Bei Einheitspreisen: J zulässig $\iff$ J Clique; also Erlös $\geq k$ $\iff$ $K_k \sqsubseteq C$. (b) Bei ganzzahligen Preisen: Ersetze Knoten j durch p_j Zwillinge, die untereinander eine Clique bilden und genau mit den Zwillingen kompatibler Gebote verbunden sind. Dann gilt: maximaler Erlös = maximale Cliquengröße im Graphen C' mit $\sum_j p_j$ Knoten (pseudopolynomiell).

Beweis. (a) Paarweise Konfliktfreiheit ist die Cliqueneigenschaft. (b) Ist J zulässig, bilden alle Zwillinge von J eine Clique der Größe $\sum_J p_j$. Ist Q Clique in C', seien J die Gebote, aus denen Q Zwillinge enthält; verschiedene $j, k \in J$ haben adjazente Zwillinge, sind also kompatibel; $|Q| \leq \sum_J p_j$. Die Äquivalenz folgt. □

Beispiel 4.4. Güter A, B, C; Gebote $b_1 = \{A,B\}$, $b_2 = \{B,C\}$, $b_3 = \{C\}$, $b_4 = \{A\}$, Preise 1. Kompatibilitätskanten: $\{b_1,b_3\}, \{b_2,b_4\}, \{b_3,b_4\}$; keine Dreiecksclique, maximaler Erlös 2. Im JSON-Format des Solvers: `"G"`: Knoten $b_1..b_4$, die drei Kanten; `"H"`: K_2 (ja) bzw. K_3 (nein).

Satz 4.5 (SAT-Kodierung mit binären Preisen)**.** [U] Variablen y_j („Gebot j gewinnt“); Klauseln $(\neg y_j \vee \neg y_k)$ für alle $j < k$ mit $S_j \cap S_k \neq \emptyset$; dazu der Schaltkreis aus Lemmata 2.6, 2.5 für $\sum_j p_j y_j \geq B$. Die entstehende CNF φ_B hat Größe $O(n^2 + nL')$ und ist genau dann erfüllbar, wenn der Erlös B erreichbar ist; die y-Belegung ist das Gewinnerset.

Beweis. Die Konfliktklauseln sind äquivalent zur Zulässigkeit; der Summierer/Vergleicher nach Lemma 2.4 zur Erlösbedingung. □

Korollar 4.6. [K]Gilt (E_T), so wird der optimale Erlös mit $\leq L' + n + 2$ Aufrufen auf Formeln der Größe $O(n^2 + nL')$ bestimmt. Unter (S_3): Laufzeit $O\big((L'+n)(n^2+nL')^3\big)$, unter ($\mathrm{E}_{\mathrm{poly}}$) polynomiell.

4.1 Vickrey-Clarke-Groves-Mechanismus

Sei Ω eine endliche Ergebnismenge, $v_i : \Omega \to \mathbb{R}_{\geq 0}$ die Bewertung von Agent $i \in \{1..N\}$ (quasilinearer Nutzen).

Definition 4.7 (VCG)**.** Wähle $\omega^\star \in \arg\max_\omega \sum_i v_i(\omega)$ bei gemeldeten v. Zahlung von i: $t_i = \max_\omega \sum_{j\neq i} v_j(\omega) - \sum_{j\neq i} v_j(\omega^\star)$.

Satz 4.8 (Wahrheitsmäßigkeit, Rationalität)**.** [U] Ehrliches Melden ist dominante Strategie, ex post individuell rational, und $t_i \geq 0$ [13, 14, 15].

Beweis. Nutzen von i mit wahrer Bewertung v_i bei Ergebnis ω': $v_i(\omega') - t_i = \big[v_i(\omega') + \sum_{j\neq i} v_j(\omega')\big] - h_i$ mit $h_i = \max_\omega \sum_{j\neq i} v_j$, unabhängig von is Meldung. Ehrliches Melden wählt ω' als Maximierer des Klammerausdrucks, kein Ergebnis ist besser. IR: $v_i(\omega^\star) + \sum_{j\neq i} v_j(\omega^\star) - h_i \geq v_i(\omega^{-i}) \geq 0$ mit ω^{-i} Maximierer von $\sum_{j\neq i} v_j$, da $\omega^\star$ die Gesamtsumme maximiert. $t_i \geq 0$ nach Definition von h_i. □

Satz 4.9 (VCG ist berechenbar mit $N+1$ Optimierungen)**.** [K] Bieten die Agenten in der XOR-Bietersprache (Gebote (S_j, p_j), je Agent höchstens eines gewinnt: zusätzliche Konfliktklauseln $(\neg y_j \vee \neg y_k)$ für Gebote desselben Agenten), so sind $\omega^\star$ und alle t_i mit $N+1$ WDP-Optimierungen bestimmbar. Unter (E_T): $(N+1)(L'+n+2)\,T(O(n^2+nL'))$; unter ($\mathrm{E}_{\mathrm{poly}}$) polynomiell.

Beweis. $\max_\omega \sum_j v_j$ und für jedes i das h_i ohne is Gebote sind WDP-Instanzen (Satz 4.5 plus Satz 2.11); $\sum_{j\neq i} v_j(\omega^\star)$ folgt aus $\omega^\star$. □

Bemerkung 4.10 (Grenzen)**.** Der Engpass „Auswertung“ entfällt, nicht aber: (i) *Kommunikation*: für allgemeine Bewertungen sind schon bei zwei Agenten und L Gütern exponentiell viele Preise/Bits nötig [16]; (ii) die bekannten Schwächen von VCG (niedrige Erlöse, Kollusion) [17]; (iii) näherungsweise Löser zerstören die Wahrheitsmäßigkeit [18], ein exakter Löser stellt sie wieder her.

5 Frequenzzuteilung und Tauschmärkte

5.1 Station Repacking (FCC-Anreizauktion)

Die FCC-Anreizauktion (2016/17) erlöste 19,8 Mrd. USD; 10,05 Mrd. USD gingen an 175 Rundfunkveranstalter. Zentral war ein SAT-basierter Zulässigkeitstester (SATFC), der bei jedem Preisangebot laufen musste [19, 20].

Definition 5.1 (Repacking)**.** Stationen S, Kanalmengen D_s, verbotene Paare $I \subseteq \{(s,c,s',c') : s \neq s'\}$. Gesucht $\gamma(s) \in D_s$, sodass kein $(s,\gamma(s),s',\gamma(s')) \in I$.

Satz 5.2 (Kodierung und NP-Vollständigkeit)**.** [U] Variablen $x_{s,c}$; Klauseln $\bigvee_{c\in D_s} x_{s,c}$; $(\neg x_{s,c} \vee \neg x_{s,c'})$ für $c \neq c'$; $(\neg x_{s,c} \vee \neg x_{s',c'})$ für $(s,c,s',c') \in I$. Die CNF ist genau dann erfüllbar, wenn ein zulässiges Repacking existiert. Das Problem ist NP-vollständig.

Beweis. ($\Rightarrow$) Aus einer Belegung: Mindestens- und Höchstens-eins-Klauseln liefern genau einen Kanal $\gamma(s)$; die Verbotsklauseln schließen verbotene Paare aus. ($\Leftarrow$) Setze $x_{s,\gamma(s)} = 1$, sonst 0. Härte: k-Färbbarkeit eines Graphen G (Karp) ist der Fall $D_s = \{1..k\}$, $I = \{(u, c, v, c) : \{u, v\} \in E(G), c \leq k\}$; Mitgliedschaft in NP trivial. □

Bemerkung 5.3. Praktisch wurde SATFC mit Portfolios, Dekomposition und Caching betrieben, ohne Worst-Case-Garantie [20, 19]. Das ist ein Hinweis auf instanzabhängige Effizienz, nicht auf ($\mathrm{E}_{\mathrm{poly}}$).

5.2 Tauschmärkte (Nierenaustausch)

Satz 5.4 (Zyklenpackung als WDP)**.** [U] Gegeben ein Digraph $D = (V, A)$ mit Gewichten und eine Zyklenlängenschranke ℓ. Sei $\mathcal{C}$ die Menge der Zyklen der Länge $\leq \ell$ ($|\mathcal{C}| \leq n^{\ell+1}$). Ein Austausch ist eine Menge knotendisjunkter Zyklen; das ist ein WDP mit Gütern V und Geboten $(V(c), w(c))_{c\in\mathcal{C}}$. Für festes ℓ ist die Kodierung von Satz 4.5 polynomiell; für $\ell \geq 3$ ist das Problem NP-schwer [21].

Beweis. Knotendisjunkte Zyklen $\Leftrightarrow$ paarweise disjunkte Knotenmengen; der Rest ist Definition und Zählung (höchstens $\sum_{i\leq\ell} n^i$ Folgen). Härte zitiert. □

6 Produktion, Logistik und Konfiguration

6.1 Terminplanung

Satz 6.1 (Einmaschinenplanung mit Freigabe- und Endterminen)**.** [U] Jobs j mit $r_j, d_j, p_j \in \mathbb{N}$; gesucht Starts $s_j \in W_j = [r_j, d_j - p_j]$ mit paarweise disjunkten Intervallen $[s_j, s_j + p_j)$. Variablen $z_{j,t}$, $t \in W_j$; Klauseln: je Job mindestens/höchstens ein Startpunkt; $(\neg z_{j,t} \vee \neg z_{k,t'})$ für $j < k$ mit $t < t' + p_k \wedge t' < t + p_j$. Erfüllbar $\iff$ zulässiger Plan; Größe $O(\sum_j |W_j|^2 + n^2T^2)$, $T = \max d_j$. Das Problem ist stark NP-vollständig [22].

Beweis. Zwei Intervalle $[t, t+p_j)$ und $[t', t'+p_k)$ überlappen genau dann, wenn $t < t'+p_k$ und $t' < t + p_j$. Der Rest wie in Satz 5.2. □

Zielfunktionen wie $\sum_j w_j(s_j + p_j) = \sum_{j,t} w_j(t + p_j)z_{j,t}$ sind linear in z und fallen unter Lemma 2.6; Minimierung gemäß Definition 2.1.

6.2 Produktkonfiguration

Produktdokumentation Φ sei eine CNF über Optionen O (Beispiel: Mercedes-Benz-Konfiguration [23]).

Satz 6.2 (Klassifikation aller Optionen)**.** [K] Option o ist *nie baubar* $\iff$ $\Phi \wedge o$ unerfüllbar; *zwingend* $\iff$ $\Phi \wedge \neg o$ unerfüllbar; ein Kundenwunsch c (Literalmenge) ist *realisierbar* $\iff$ $\Phi \wedge c$ erfüllbar. Alle $|O|$ Optionen werden mit $2|O|$ Entscheideraufrufen klassifiziert. Minimale Wunschverletzung ist ein WOP (Satz 2.11).

Beweis. Unmittelbar aus der Definition der Modelle von Φ. Wunschverletzung: $U(x) = \sum_k w_k[\text{Wunsch } k \text{ erfüllt}]$, $F = \Phi$. □

Bemerkung 6.3. Die *Anzahl* gültiger Konfigurationen ist #P-vollständig (Satz 3.8), SAT genügt dafür nicht; die Praxis nutzt eigene Modellzähler.

6.3 Standortplanung

Satz 6.4 (Uncapacitated Facility Location)**.** [U] Fabriken i mit Öffnungskosten f_i, Kunden j, Zuordnungskosten $c_{ij} \in \mathbb{N}$. Variablen y_i, x_{ij}; Klauseln $\bigvee_i x_{ij}$ und $(\neg x_{ij} \vee y_i)$; Kostenbedingung $\sum_i f_i y_i + \sum_{ij} c_{ij} x_{ij} \leq B$ (Lemma 2.6, 2.5). Erfüllbar $\iff$ Kosten $\leq B$ erreichbar. NP-schwer.

Beweis. Eine Lösung ordnet jedem Kunden eine offene Fabrik zu; mehrfaches $x_{ij} = 1$ erhöht nur die Kosten, jede erfüllende Belegung liefert eine Zuordnung mit höchstens diesen Kosten, und die optimale Zuordnung ist eine Belegung. Härte: Aus Set Cover (Mengen S_i, Elemente j, Budget k): $f_i = 1$, $c_{ij} = 0$ falls $j \in S_i$, sonst $k+1$. Kosten $\leq k$ sind nur mit höchstens k offenen Fabriken und ohne teure Zuordnung möglich, d. h. mit einer Überdeckung der Größe $\leq k$; die Umkehrung ist klar. □

Bemerkung 6.5 (Weitere Felder)**.** Kardinalitätsbeschränkte Portfolioauswahl ist NP-schwer [24]; Lieferketten-, Touren- und Schichtplanung besitzen ganzzahlige Kodierungen derselben Bauart (WOP).

7 Spiele, Gleichgewichte und kollektive Entscheidungen

Satz 7.1 (Näherungsgleichgewichte liegen in FNP)**.** [U] In einem endlichen k-Personen-Spiel mit rationalen Auszahlungen $|u_i(a)| \leq U$ und $\varepsilon > 0$ existiert ein ε-Nash-Gleichgewicht mit rationalen Wahrscheinlichkeiten polynomieller Bitlänge, das in Polynomialzeit verifizierbar ist.

Beweis. Nash [25] liefert ein exaktes Gleichgewicht x^*. Wähle rationale x' mit $\|x'_j - x^*_j\|_1 \leq \eta = \varepsilon/(2kU)$ (Rundung auf Nenner $\lceil |A_j|/\eta \rceil$). Hybridargument: $|u_i(x) - u_i(x')| \leq k\eta U$ und $|u_i(a'_i, x_{-i}) - u_i(a'_i, x'_{-i})| \leq (k-1)\eta U$. Damit $u_i(a'_i, x'_{-i}) \leq u_i(a'_i, x^*_{-i}) + (k-1)\eta U \leq u_i(x^*) + (k-1)\eta U \leq u_i(x') + (2k-1)\eta U \leq u_i(x') + \varepsilon$. Die Prüfung der Bedingung ist eine endliche Rechnung in Polynomialzeit. □

Korollar 7.2 (Nash unter ($\mathrm{E}_{\mathrm{poly}}$))**.** [K]Gilt ($\mathrm{E}_{\mathrm{poly}}$), so sind ε-Gleichgewichte (Eingabe ε binär) in Polynomialzeit berechenbar (Satz 2.15). Das Problem ist PPAD-vollständig [26, 27], also folgt PPAD $\subseteq$ FP [28, 29].

Bemerkung 7.3. Die Existenz von Gleichgewichten mit Zusatzeigenschaften (z. B. Wohlfahrt $\geq w$, 2 Spieler) ist NP-vollständig [30, 31], in NP und damit unter ($\mathrm{E}_{\mathrm{poly}}$) polynomiell; *Zählen* von Gleichgewichten bleibt #P-hart [31]. Für knappe (Schaltkreis-)Spiele ist die Existenz reiner Gleichgewichte Σ_2^p-vollständig [32] und kollabiert nach Satz 3.10.

7.1 Soziale Wahl

Satz 7.4 (Rangaggregation (Kemeny))**.** [U] Kandidaten $1..q$, w_{ab} = Zahl der Wähler mit a vor b. Variablen o_{ab} $(a < b)$, $o_{ba} := \neg o_{ab}$; Klauseln $(\neg o_{ab} \vee \neg o_{bc} \vee o_{ac})$ für alle paarweise verschiedenen a, b, c. Die Modelle sind genau die linearen Ordnungen; die Kosten $\sum_{a \neq b} w_{ab}[o_{ba}]$ sind linear (Lemma 2.6). Das Problem ist NP-schwer [33].

Beweis. Ein Turnier ist genau dann transitiv, wenn es keinen gerichteten 3-Zyklus gibt: Aus $a \to b \to c$ und $c \to a$ entstünde ein Zyklus, also gilt $a \to c$. Die Klauseln verbieten genau diese Zyklen; ein transitives Turnier ist eine strikte lineare Ordnung. Die Kosten sind die Kendall-Tau-Distanz. □

Satz 7.5 (Manipulation ist in NP)**.** [K] Sei f eine in Polynomialzeit berechenbare Wahlregel. Das Problem „Gibt es einen Stimmzettel b des Wählers i mit $f(b, b_{-i}) =$ Zielergebnis?“ liegt in NP; unter ($\mathrm{E}_{\mathrm{poly}}$) in P.

Beweis. Zeuge b, Prüfung durch Auswertung von f; Satz 2.14. □

Damit entfiele die von [34] genutzte Schutzwirkung rechnerischer Härte; der Satz von Gibbard–Satterthwaite [35, 36] bleibt unberührt.

8 Kryptographie, Zahlungsverkehr und Sicherheit

Definition 8.1 (Einwegfunktion)**.** Eine längenerhaltende, in Polynomialzeit berechenbare Funktion f ist *einweg*, wenn für jeden probabilistischen Polynomialzeitalgorithmus A gilt: $\Pr_x[f(A(f(x))) = f(x)] \le 1/\operatorname{poly}(|x|)$.

Satz 8.2 (Keine Einwegfunktionen unter ($\mathrm{E}_{\mathrm{poly}}$))**.** [K] Gilt ($\mathrm{E}_{\mathrm{poly}}$), so existiert keine Einwegfunktion.

Beweis. Für f sei $R(y, x) :\iff f(x) = y$ (polynomiell beschränkt, in Polynomialzeit entscheidbar). Nach Satz 2.15 findet ein deterministischer Polynomialzeitalgorithmus zu jedem y im Bild ein x mit $f(x) = y$. Er invertiert f mit Wahrscheinlichkeit 1. □

Korollar 8.3. [K]Da die Existenz u. a. von sicheren Signaturen, Pseudozufallsgeneratoren und Verschlüsselung mit Schlüsselkürzung die Existenz von Einwegfunktionen impliziert [37, 39, 38], wären unter ($\mathrm{E}_{\mathrm{poly}}$) alle diese Primitive in der asymptotischen Sicherheitsdefinition unsicher: Schlüsselsuche bei bekanntem Klartext ist ein FNP-Problem.

Bemerkung 8.4 (Ökonomische Lesart)**.** Zahlungsverkehr, Signaturen, TLS und Proof-of-Work-Verfahren stützen sich auf solche Annahmen; Proof-of-Work ist im asymptotischen Modell eine FNP-Suche. Informationstheoretisch sichere Verfahren (z. B. One-Time-Pad [40]) bleiben unberührt. Ein effizienter Solver wäre also nichtmonoton: Gewinne in Kapiteln 3–7 stehen Ersatzkosten der Sicherheitsinfrastruktur gegenüber (Kapitel 9). Die heutige Praxis zeigt, dass praktische SAT-Löser diese Primitive nicht brechen; das spricht gegen ($\mathrm{E}_{\mathrm{poly}}$), nicht für Entwarnung im Szenario.

9 Wohlfahrtsmodell und Informationsgrenzen

Definition 9.1 (Rechenlücke)**.** Agent i erreicht mit begrenzten Rechenmitteln den Wert $A_i \le \mathrm{opt}_i$; die Lücke ist $g_i = (\mathrm{opt}_i - A_i)/\mathrm{opt}_i \in [0, 1]$. Gesamtoutput $Y = \sum_i \mathrm{opt}_i(1 - g_i)$, Optimum $O = \sum_i \mathrm{opt}_i$.

Satz 9.2 (Wohlfahrtsgewinn)**.** [U] Machen NP-artige Entscheidungen den Wertanteil α von O aus, bei wertgewichteter mittlerer Lücke $\bar{\gamma}$ und Lücke 0 sonst, so ist der relative Zugewinn durch exakte Lösung

$$\frac{O - Y}{Y} = \frac{\alpha\bar{\gamma}}{1 - \alpha\bar{\gamma}}.$$

Beweis. $O - Y = \alpha\bar{\gamma}O$, $Y = O(1 - \alpha\bar{\gamma})$. □

Beispiel 9.3 (rein illustrativ, keine Schätzung)**.** $\alpha = 0{,}10$, $\bar{\gamma} = 0{,}05$ ergibt $0{,}005/0{,}995 \approx 0{,}50\,\%$. Die Parameter $\alpha, \bar{\gamma}$ sind empirisch zu bestimmen (Kapitel 11); aus dieser Arbeit folgt kein Zahlenwert. Abzuziehen sind Ersatzkosten der Sicherheitsinfrastruktur (Kapitel 8).

Satz 9.4 (Kommunikationsgrenze)**.** [Z] Zur Zuteilung L unteilbarer Güter an zwei Agenten mit allgemeinen Bewertungen ist für wertmaximale Zuteilung exponentielle Kommunikation in L nötig; ein Preis je Bündel $(2^L - 1)$ muss offenbart werden [16].

Rechenleistung ersetzt also nicht das Wissensproblem [41]: Ein exakter Solver löst „Gegeben die Daten, plane optimal“ (ganzzahlig NP-schwer, linear polynomiell [44]), nicht „Wie gelangen die verteilten Daten zum Planer?“ [42, 43]. Das Rechenargument gegen zentrale Planung würde geschwächt, das Informations- und Anreizargument bleibt bestehen. Ebenso senkt $(\mathrm{E}_{\mathrm{poly}})$ die rechnerischen Transaktionskosten im Sinne von [45] für Vertragsgestaltung in NP, nicht aber Verhandlungs- und Informationskosten. Marktstrukturelle Folgen (Konzentration bei Zugang zum Solver) sind eine offene, nicht formalisierte Frage.

10 Experimente mit Subgraph-SAT-Solver

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

10.1 Ziel und Versuchsaufbau

Dieses Kapitel ergänzt die Reduktionsbeweise um eine ausführbare Messreihe. Sie veranschaulicht, wie ökonomische Modelle aus dem Repository in SAT-Abfragen übersetzt und vom nativen C++-Projekt `subgraph-sat-solver` beantwortet werden. Der Versuchsplan umfasst sechs Problemfamilien: Winner Determination in kombinatorischen Auktionen, Frequenzzuteilung als Repacking, gewichtete Einmaschinenplanung, unkapazitiertes Standortproblem, Nierenaustausch als Zyklenpackung und Kemeny-Rangaggregation. Der hier ausgewertete Datenstand enthält Ergebnisse für zwei dieser Familien; die übrigen vier werden nicht als durchgeführte Messungen dargestellt. Die generierten Instanzgrößen variieren je Familie; mehrere feste Zufallsseeds verhindern, dass eine einzelne günstige Instanz den Verlauf bestimmt.

Für jede erzeugte Instanz wird derselbe ökonomische Löser einmal mit `NativeBackend` (C++-Bibliothek `satlib` über den C-ABI-Shim und das externe `csubgraph`-Programm) und einmal mit dem unabhängigen Python-`DpllBackend` ausgeführt. Die Python-Ausführung ist eine Referenz und ausdrücklich kein Ersatz oder Fallback für den nativen Solver. Der Lauf bricht ab, wenn CMake, Shim, Subgraph-Programm oder eine native SAT-Antwort fehlschlagen. Es werden somit keine als C++-Ergebnisse ausgegebenen Python-Fallback-Messungen erzeugt.

Die ökonomischen Optimierungsroutinen verwenden wiederholt Entscheidungsabfragen, um optimale Zielfunktionswerte zu bestimmen und Lösungen selbstreduzierend zu rekonstruieren. Die Zeitangaben umfassen den vollständigen Aufruf des jeweiligen ökonomischen Lösers einschließlich dieser Abfragen. Zusätzlich erfasst der Messadapter

die Anzahl der SAT-Aufrufe, die größte beobachtete einzelne CNF-Abfrage und für den nativen Solver die aufsummierten Phasenzeiten für Reduktion, Subgraph-Entscheidung und Extraktion. Pro Instanz muss der vom Modul gemeldete Aufrufzähler mit der tatsächlich beobachteten Zahl übereinstimmen. Der native und der Referenzlauf müssen denselben Zielfunktionswert liefern; ausgegebene Lösungen werden außerdem anhand der jeweiligen ökonomischen Zulässigkeitsbedingungen nachgeprüft.

10.2 Instanzfamilien

Auktionen. Zufällig erzeugte Bündelgebote mit überlappenden Gütern und XOR-Bietern; maximiert wird der Gesamterlös bei zulässiger Gewinnerauswahl. Die Instanzgröße ist die Gebotszahl.

Frequenzzuteilung. Repacking-Instanzen mit drei Kanälen und zufälligen Interferenzbedingungen. Jede erzeugte Instanz besitzt eine konstruierte zulässige Färbung, sodass der Test eine echte, überprüfte Zuteilung statt eines bloßen UNSAT-Ergebnisses verlangt.

Terminplanung. Jobs haben Freigabezeiten, Bearbeitungsdauern und Gewichte. Minimiert wird die gewichtete Fertigstellungszeit unter Einmaschinen- und Zeitfensterbedingungen.

Standortplanung. Zufällige Eröffnungs- und Zuordnungskosten für Fabriken und Kunden; minimiert werden die gesamten Kosten eines zulässigen Standort- und Zuordnungsplans.

Nierenaustausch. Gerichtete Spender-Empfänger-Graphen werden in knotendisjunkte Zyklen der Länge höchstens drei zerlegt. Maximiert wird die Summe der Gewichte der ausgeführten Austausche.

Kemeny-Rangaggregation. Zufällige vollständige Stimmzettel werden zu paarweisen Präferenzgewichten aggregiert. Gesucht wird eine Konsensrangfolge mit minimaler gewichteter Kendall-Tau-Kostenfunktion.

Die Instanzen sind synthetisch und dienen der technischen Veranschaulichung, nicht als empirische Stichprobe realer Auktionen, Frequenzmärkte, Produktionsbetriebe oder Transplantationsnetzwerke. Das vermeidet eine Interpretation der Zufallsdaten als beobachtete wirtschaftliche Effekte. Die Generierung ist deterministisch: Startseed, Instanzseed und Wiederholungsindex sind in jeder JSON-Datei enthalten.

10.3 Messwerte

Der gegenwärtig gespeicherte Datenstand umfasst zwei abgeschlossene Problemfamilien: Auktionen und Repacking. Für jede der beiden Familien liegen vier Größenstufen mit je drei Wiederholungen vor, insgesamt also 24 Instanzläufe. Die weiteren vier im Skript vorgesehenen Familien (Terminplanung, Standortplanung, Nierenaustausch

und Kemeny-Rangaggregation) sind in den vorliegenden Ergebnisdateien noch nicht enthalten und werden deshalb nicht als gemessene Resultate ausgewertet. Die Rohdaten und die aus ihnen berechneten Kennzahlen stehen in `src/results/auctions.json` und `src/results/repacking.json`.

Tabelle 1 gibt für jede Größe die Medianlaufzeit des nativen Backends und des Python-DPLL an. Native Laufzeiten sind Ende-zu-Ende-Zeiten erfolgreicher Läufe; bei sechs Geboten wird daher der Median der zwei erfolgreichen nativen Wiederholungen ausgewiesen. Die DPLL-Spalte enthält den Median aller drei entsprechenden Läufe. Die CNF-Größe ist der Median der jeweils größten einzelnen SAT-Abfrage in einer Optimierung, nicht die Größe der Summe aller Abfragen.

Problemfamilie	Größe	Nat. Median (ms)	DPLL Median (ms)	SAT-Aufrufe (Median)	Nat. Erfolge (/3)
Auktion	3 Gebote	1080,8	2,82	11	3
Auktion	4 Gebote	1415,7	6,24	13	3
Auktion	5 Gebote	1310,5	7,38	14	3
Auktion	6 Gebote	4709,7	15,75	15	2
Repacking	3 Stationen	47,3	0,12	1	3
Repacking	4 Stationen	50,1	0,22	1	3
Repacking	5 Stationen	50,1	0,18	1	3
Repacking	6 Stationen	54,0	0,32	1	3

Tabelle 1: Medianlaufzeiten und native Abschlusszahlen je Familiengröße. Die kleinformatigen DPLL-Zeiten sind in Millisekunden angegeben; sie sind nicht mit der Subgraph-Startlatenz des nativen Pfads gleichzusetzen.

Bei Auktionen steigen die Medianwerte der größten Einzel-CNF-Abfrage von 21 Variablen, 64 Klauseln und 163 Literalvorkommen bei drei Geboten auf 89, 295 und 774 bei sechs Geboten. Die mediane Zahl der SAT-Aufrufe nimmt gleichzeitig von 11 auf 15 zu: Neben den steigenden Formelkosten wird also auch die Optimierungsschleife häufiger aufgerufen. Die Laufzeit reagiert allerdings nicht glatt auf die Eingabegröße. Bei fünf Geboten ist der Median 1,31 Sekunden, bei sechs Geboten 4,71 Sekunden unter den erfolgreichen Läufen. Schon bei fünf Geboten streuen die drei nativen Laufzeiten stark: eine Wiederholung benötigt etwa 13,01 Sekunden, während die beiden anderen etwa 1,10 und 1,31 Sekunden benötigen. Ein günstiger Median verdeckt somit einzelne schwierige Instanzen.

Bei Repacking wächst die größte Einzel-CNF über dieselben Größenstufen von 9/12/27 (Variablen/Klauseln/Literalvorkommen) auf 18/39/84. Es genügt jeweils ein SAT-Aufruf; die native Ende-zu-Ende-Zeit steigt in der kleinen Stichprobe von ungefähr 47 auf 54 Millisekunden. Die Messung wird hier weitgehend von der festen Aufruf- und Prozessstartlatenz des Subgraph-Programms bestimmt, nicht von einer großen Suchinstanz.

Über die 23 erfolgreich beantworteten nativen Fälle stimmen die optimalen Zielfunktionswerte mit Python-DPLL überein; alle zurückgegebenen ökonomischen Lösungen bestanden die im Experiment ausgeführte Zulässigkeitsprüfung. Der verbleibende Fall ist eine Auktion mit sechs Geboten: Der native Aufruf lieferte einen Fehler durch Überschreitung des Subgraph-Zeitlimits, bevor ein Ergebnis vorlag. Der gesamte ökonomische Aufruf dauerte dabei 96,96 Sekunden und scheiterte während des

achten SAT-Aufrufs. Das gesetzte Limit von 60 Sekunden gilt pro Subgraph-Instanz, nicht als Gesamtlimit für die Folge von SAT-Aufrufen einer Optimierung. Ein Timeout ist keine UNSAT-Antwort und darf nicht als gelöster Fall gewertet werden.

In den erfolgreichen Läufen dominiert die Subgraph-Phase die vom nativen Solver ausgewiesenen Phasenzeiten: Der Median der aufsummierten Subgraph-Zeit beträgt rund 1,09 Sekunden bei Auktionen und 49,48 Millisekunden bei Repacking. Die entsprechenden Reduktionsmediane betragen etwa 210 Millisekunden beziehungsweise 0,29 Millisekunden; die Extraktion bleibt mit etwa 0,23 beziehungsweise 0,01 Millisekunden klein. Bezogen auf die Summe der aggregierten Phasenmediane entfallen damit rund 84 % bei Auktionen und 99 % bei Repacking auf den Subgraph-Aufruf. Bei Auktionen ist die Reduktion durch die wiederholte, wachsende CNF ebenfalls messbar. Diese Phasenwerte summieren sich über die SAT-Aufrufe einer Optimierung und sind nicht die Laufzeit eines einzelnen Aufrufs.

Auf diesen kleinen Instanzen ist der native vollständige Pfad langsamer als der direkte Python-DPLL: Der Median der gepaarten nativen/DPLL-Laufzeitverhältnisse beträgt etwa 232 bei Auktionen und 256 bei Repacking. Das ist kein Widerspruch zur Verwendung der C++-Bibliothek: Der native Pfad führt für jede Abfrage Reduktion, Graphserialisierung und einen externen Subgraph-Prozess aus; DPLL löst dieselbe kleine CNF unmittelbar im Prozess. Die vorliegenden Größen sind zu klein, um mögliche Vorteile auf größeren oder strukturell andersartigen Formeln abzubilden. Der Vergleich belegt daher weder einen allgemeinen Leistungsvergleich der Algorithmen noch einen Vorteil des nativen Backends.

Abbildung 2 zeigt die Medianlaufzeiten logarithmisch. Ein rotes Kreuz markiert die nicht abgeschlossene Auktionsinstanz; der Punkt ist ihre verstrichene Fehlerzeit, kein erfolgreicher Lösungswert. Abbildung 3 stellt die größte Einzel-CNF der jeweiligen Optimierung dar.

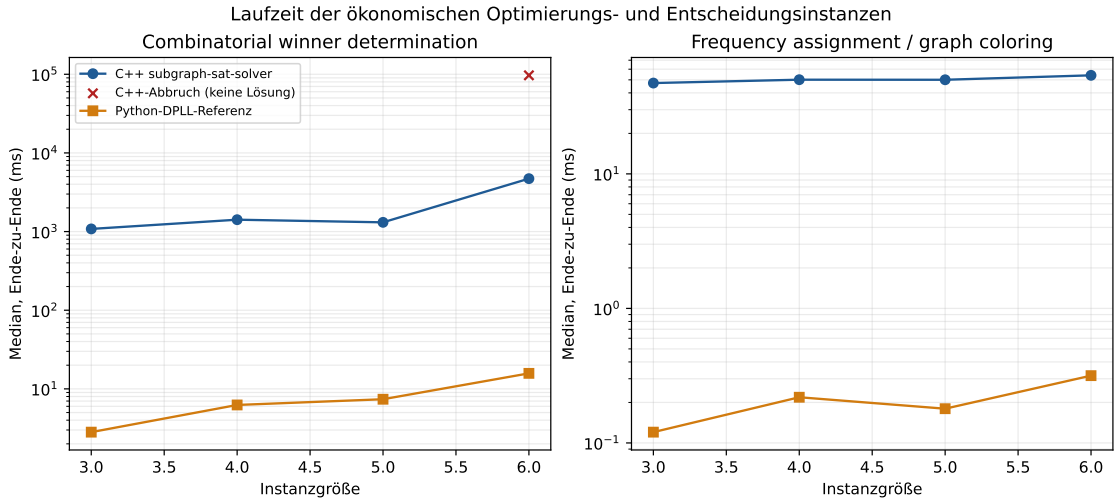


Abbildung 2: Median der Ende-zu-Ende-Laufzeit in den gemessenen Auktionen und Repacking-Instanzen. Das Kreuz kennzeichnet einen abgebrochenen nativen Lauf.

Abbildung 4 zerlegt die gemessene Arbeit des nativen Solvers nach den von ihm berichteten Phasen. Diese Werte summieren sich über die SAT-Aufrufe einer ökonomischen Optimierung. Sie sind daher nicht direkt mit der Laufzeit eines einzelnen SAT-Aufrufs gleichzusetzen; der äußere Ende-zu-Ende-Wert enthält außerdem Python-Modellierung, ctypes-Übergänge und Selbstreduktion.

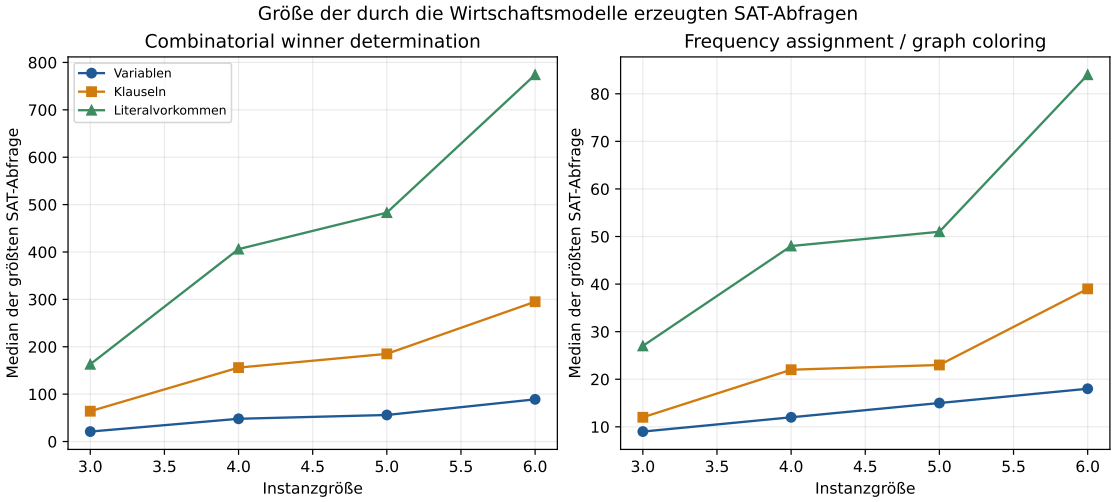


Abbildung 3: Größe der größten SAT-Abfrage je Instanz, gemessen in Variablen, Klauseln und Literalvorkommen.

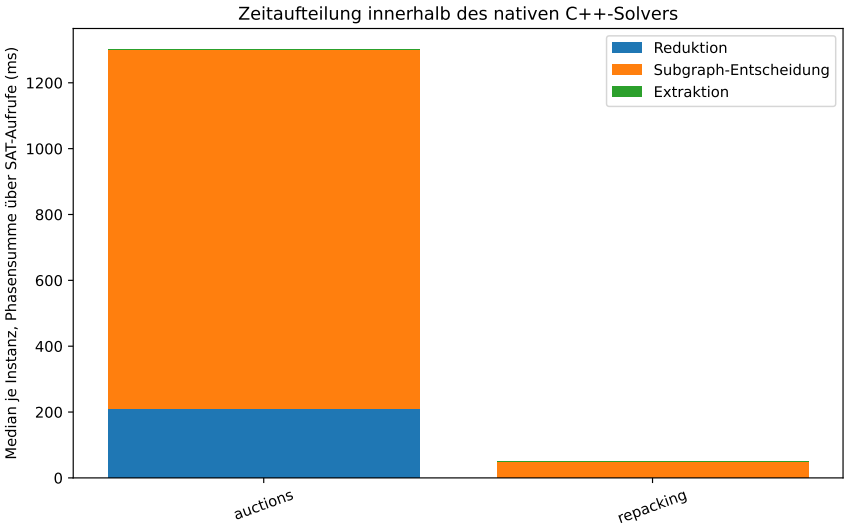


Abbildung 4: Median der aufsummierten nativen Reduktions-, Subgraph- und Extraktionszeiten je erfolgreicher Instanz.

Abbildung 5 ergänzt die Laufzeitgrafik um die Abschlussquote: Bei Auktionen wurden 11 von 12 nativen Fällen abgeschlossen, bei Repacking 12 von 12. Alle Rohmessungen einschließlich Seeds, Instanzgrößen, Zielfunktionswerten, Aufrufzahlen, CNF-Größen und nativen Phasenzeiten sind einzeln als JSON unter `src/results/` abgelegt. Die Abbildungen werden aus diesen Daten durch `src/experiments.py` mit Matplotlib erzeugt.

Die Messungen wurden unter Windows 11 mit Python 3.13.16, dem MinGW-C++-Compiler und den benachbarten Checkouts von `subgraph-sat-solver` und `csubgraph` ausgeführt. Die Instanzen sind durch feste Seeds reproduzierbar; Hardwaremodell und weitere Systemlast sind in den Ergebnisdateien jedoch nicht dokumentiert. Die Laufzeiten sind daher als Messwerte dieser konkreten Umgebung und nicht als universelle Kennzahlen zu lesen. Der native Pfad lief im exakten Modus; die unabhängige Referenz war der Python-DPLL aus dem Repository, kein optimierter CDCL-Solver.

Reproduktion aus dem Repository-Wurzelverzeichnis:

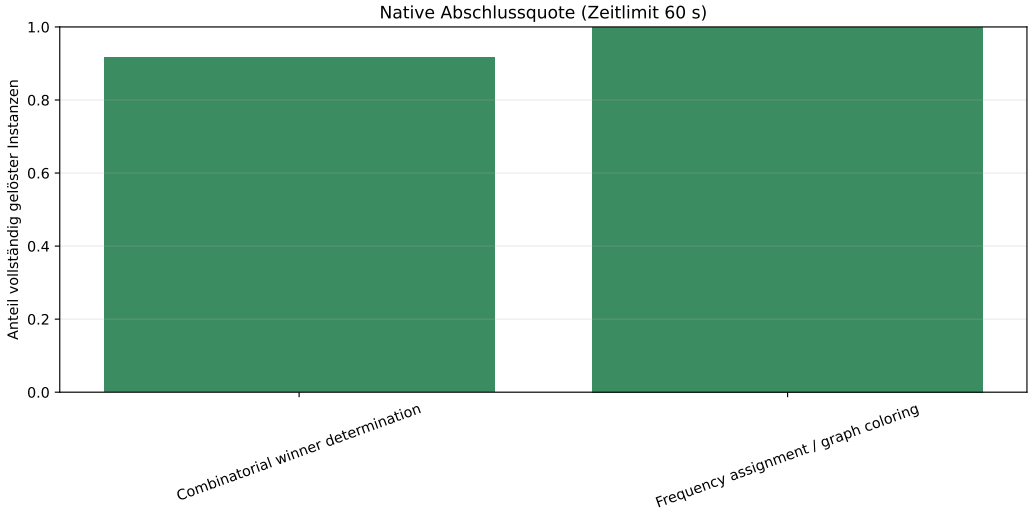


Abbildung 5: Anteil der nativen Instanzläufe mit vollständig zurückgegebenem Ergebnis (Zeitlimit je Subgraph-Entscheidung: 60 Sekunden).

```
python -m pip install -e .
python -m src.experiments --seed 20261013 --repetitions 3 --timeout-sec 60
```

Der Lauf baut den Shim gegen den benachbarten Checkout `../subgraph-sat-solver` und erwartet `../csubgraph/build/subgraph.exe`; alternative Orte sind über `--solver-dir` und `--subgraph-binary` angebbar. Jede abgeschlossene Instanz wird sofort in ihrer JSON-Datei gespeichert. Ein Timeout wird als Fehler protokolliert und nicht in UNSAT umgedeutet. Der vollständige Lauf erzeugt JSON-Dateien pro Familie und die PDF-Abbildungen unter `science/figures/`; nicht durchgeführte Familien dürfen dabei nicht als gemessene Resultate interpretiert werden.

10.4 Interpretation und Grenzen

Die Ergebnisse sind eine technische Machbarkeits- und Kostenaufnahme für zwei Modulbeispiele, keine repräsentative ökonomische Stichprobe. Vier der vorgesehenen Problemfamilien fehlen noch, die Fallzahlen pro Größe betragen nur drei, und die Auktionen zeigen bereits einen Timeout sowie stark streuende Laufzeiten. Weder lässt sich daraus auf große Märkte extrapolieren noch ein allgemeiner Leistungsvergleich von C++ und Python ableiten. Insbesondere belegen die Messungen nicht die Effizienzhypothese (E_{poly}) oder eine polynomielle Laufzeit des Subgraph-Orakels. Ihr positiver Befund ist enger: Für 23 erfolgreich abgeschlossene synthetische Instanzen stimmen der native Solver und DPLL im Optimalwert überein, und die ausgegebenen Lösungen sind in den getesteten Modulen zulässig. Zugleich macht der hohe Anteil der Subgraph-Phase die Prozess- und Suchkosten des aktuellen nativen Integrationspfads sichtbar.

11 Grenzen, Messplan und Fazit

Tabelle 2 fasst die Reichweite der Ergebnisse zusammen. Die Spalte „Problemklasse“ nennt die Komplexitätsklasse, die Spalte „Beispiel“ ein ökonomisches Problem, das in dieser

Klasse liegt, und die Spalte „Wirkung von (E_{poly})“ beschreibt, ob und wie die Annahme (E_{poly}) das Problem beeinflusst. Die Tabelle ist nicht erschöpfend; sie zeigt exemplarisch, dass (E_{poly}) alle Probleme in NP und der Polynomialzeithierarchie betrifft, aber nicht Zähl- und PSPACE-Probleme, Informationsprobleme oder kryptographische Aufgaben.

Problemklasse	Beispiel	Wirkung von (E_{poly})
NP / coNP	WDP, Repacking, Planung	polynomiell (Satz 2.11, Kor. 2.12)
Σ_k^p, Π_k^p	robuste Mehrstufenplanung	polynomiell (Satz 3.10)
PPAD $\subseteq$ FNP	Nash-Näherung	polynomiell (Satz 7.1)
#P	Modell- und Gleichgewichtszählen	keine bekannte Wirkung (Satz 3.8)
PSPACE	POMDP, Spiele	keine bekannte Wirkung
Informationsprobleme	Hayek, Elicitation	keine Wirkung (Satz 9.4)
Kryptographie	Signaturen, Zahlungen	Zusammenbruch im asymptotischen Modell (Satz 8.2)

Tabelle 2: Reichweite und Grenzen der Ergebnisse.

11.1 Messplan mit dem Repository

Die Ergebnisse sind konditional; empirisch lässt sich prüfen, wie weit die Kette praktisch trägt. Vorgeschlagen: (1) Auktionsinstanzen mit dem Generator CATS [46] und Repacking-Instanzen erzeugen, nach Satz 4.5 bzw. 5.2 als DIMACS-CNF kodieren und an den Solver übergeben; (2) jede Antwort gegen einen unabhängigen Entscheider (CDCL, [47]) und per Belegungsprüfung validieren; (3) Phasenlaufzeiten (Kodierung, Reduktion, Orakel, Extraktion), Zeitüberschreitungen und Fehler getrennt berichten; (4) Skalierung der Graphgröße gegen N^3 (Lemma 2.8) auftragen. Ein exaktes Ergebnis darf nur gemeldet werden, wenn der Orakelmodus exakt ist (Grundarbeit, Abschnitt zur Orakel-Schnittstelle). Die hier angegebenen Kodierungen (Auktion, Planung, Repacking, Standort, Rangaggregation, Pareto-Algorithmus) wurden vom Verfasser an kleinen Zufallsinstanzen gegen vollständige Enumeration geprüft; das ersetzt keine Beweise, sichert aber die Formeln.

11.2 Fazit

Unter der Annahme eines exakten, effizienten SAT-Entscheiders lösen sich in Polynomialzeit alle ökonomischen Entscheidungsprobleme in NP und der gesamten Polynomialzeithierarchie, einschließlich Auktionsauswertung, VCG-Zahlungen, Planung, Konfiguration und Gleichgewichtsberechnung. Nicht gelöst werden Zähl- und PSPACE-Probleme sowie Informations- und Anreizprobleme; und die kryptographische Grundlage digitaler Märkte bräche weg. Da die Annahme P = NP entspricht, sind die Kapitel als präzise Beschreibung des Szenarios zu lesen, nicht als Nachweis, dass der Solver es herbeiführt.

Literaturverzeichnis

[1] S. A. Cook. The complexity of theorem-proving procedures. *Proc. 3rd ACM STOC*, 1971.

[2] R. M. Karp. Reducibility among combinatorial problems. 1972.

[3] C. H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.

[4] S. Arora, B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[5] L. Fortnow. The status of the P versus NP problem. *CACM* 52(9):78–86, 2009.

[6] S. Aaronson. P $\stackrel{?}{=}$ NP. In: *Open Problems in Mathematics*, Springer, 2016, 1–122.

[7] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM J. Comput.* 8(3):410–421, 1979.

[8] L. Stockmeyer. The complexity of approximate counting. *STOC* 1983, 118–126.

[9] C. H. Papadimitriou, J. N. Tsitsiklis. The complexity of Markov decision processes. *Math. Oper. Res.* 12(3):441–450, 1987.

[10] H. A. Simon. A behavioral model of rational choice. *Quarterly J. Economics* 69(1):99–118, 1955.

[11] M. H. Rothkopf, A. Pekeč, R. M. Harstad. Computationally manageable combinational auctions. *Management Science* 44(8):1131–1147, 1998.

[12] T. Sandholm. Algorithm for optimal winner determination in combinatorial auctions. *Artificial Intelligence* 135(1–2):1–54, 2002.

[13] W. Vickrey. Counterspeculation, auctions, and competitive sealed tenders. *J. Finance* 16(1):8–37, 1961.

[14] E. H. Clarke. Multipart pricing of public goods. *Public Choice* 11:17–33, 1971.

[15] T. Groves. Incentives in teams. *Econometrica* 41(4):617–631, 1973.

[16] N. Nisan, I. Segal. The communication requirements of efficient allocations and supporting prices. *J. Economic Theory* 129(1):192–224, 2006.

[17] L. M. Ausubel, P. Milgrom. The lovely but lonely Vickrey auction. In: *Combinatorial Auctions*, MIT Press, 2006.

[18] N. Nisan, A. Ronen. Algorithmic mechanism design. *Games and Economic Behavior* 35(1–2):166–196, 2001.

[19] N. Newman, A. Fréchette, K. Leyton-Brown. Deep optimization for spectrum repacking. *CACM* 61(1):97–104, 2018.

[20] A. Fréchette, N. Newman, K. Leyton-Brown. Solving the station repacking problem. *AAAI* 2016.

[21] D. J. Abraham, A. Blum, T. Sandholm. Clearing algorithms for barter exchange markets: Enabling nationwide kidney exchanges. *ACM EC* 2007.

[22] J. K. Lenstra, A. H. G. Rinnooy Kan, P. Brucker. Complexity of machine scheduling problems. *Annals of Discrete Mathematics* 1:343–362, 1977.

[23] C. Sinz, A. Kaiser, W. Küchlin. Formal methods for the validation of automotive product configuration data. *AI EDAM* 17(1):75–97, 2003.

[24] D. Bienstock. Computational study of a family of mixed-integer quadratic programming problems. *Math. Programming* 74:121–140, 1996.

[25] J. Nash. Non-cooperative games. *Annals of Mathematics* 54(2):286–295, 1951.

[26] C. Daskalakis, P. W. Goldberg, C. H. Papadimitriou. The complexity of computing a Nash equilibrium. *SIAM J. Comput.* 39(1):195–259, 2009.

[27] X. Chen, X. Deng, S.-H. Teng. Settling the complexity of computing two-player Nash equilibria. *J. ACM* 56(3), 2009.

[28] C. H. Papadimitriou. On the complexity of the parity argument and other inefficient proofs of existence. *JCSS* 48(3):498–532, 1994.

[29] N. Megiddo, C. H. Papadimitriou. On total functions, existence theorems and computational complexity. *Theoret. Comput. Sci.* 81(2):317–324, 1991.

[30] I. Gilboa, E. Zemel. Nash and correlated equilibria: Some complexity considerations. *Games and Economic Behavior* 1:80–93, 1989.

[31] V. Conitzer, T. Sandholm. New complexity results about Nash equilibria. *Games and Economic Behavior* 63(2):621–641, 2008.

[32] G. Schoenebeck, S. Vadhan. The computational complexity of Nash equilibria in concisely represented games. *ACM EC* 2006, 270–279.

[33] J. Bartholdi, C. Tovey, M. Trick. Voting schemes for which it can be difficult to tell who won the election. *Social Choice and Welfare* 6(2):157–165, 1989.

[34] J. Bartholdi, C. Tovey, M. Trick. The computational difficulty of manipulating an election. *Social Choice and Welfare* 6(3):227–241, 1989.

[35] A. Gibbard. Manipulation of voting schemes. *Econometrica* 41(4):587–601, 1973.

[36] M. Satterthwaite. Strategy-proofness and Arrow's conditions. *J. Economic Theory* 10(2):187–217, 1975.

[37] R. Impagliazzo, M. Luby. One-way functions are essential for complexity based cryptography. *FOCS* 1989.

[38] R. Impagliazzo. A personal view of average-case complexity. *Structure in Complexity Theory* 1995, 134–147.

[39] O. Goldreich. *Foundations of Cryptography, Vol. I.* Cambridge University Press, 2001.

[40] C. E. Shannon. Communication theory of secrecy systems. *Bell System Technical J.* 28(4):656–715, 1949.

[41] F. A. Hayek. The use of knowledge in society. *American Economic Review* 35(4):519–530, 1945.

[42] L. von Mises. Die Wirtschaftsrechnung im sozialistischen Gemeinwesen. *Archiv für Sozialwissenschaften* 47:86–121, 1920.

[43] L. Hurwicz. On informationally decentralized systems. In: *Decision and Organization*, North-Holland, 1972.

[44] L. G. Khachiyan. A polynomial algorithm in linear programming. *Soviet Math. Doklady* 20:191–194, 1979.

[45] R. H. Coase. The nature of the firm. *Economica* 4(16):386–405, 1937.

[46] K. Leyton-Brown, M. Pearson, Y. Shoham. Towards a universal test suite for combinatorial auction algorithms. *ACM EC* 2000, 66–76.

[47] N. Eén, N. Sörensson. An extensible SAT-solver. *SAT 2003*, 502–518.